Technical Assessment Answers

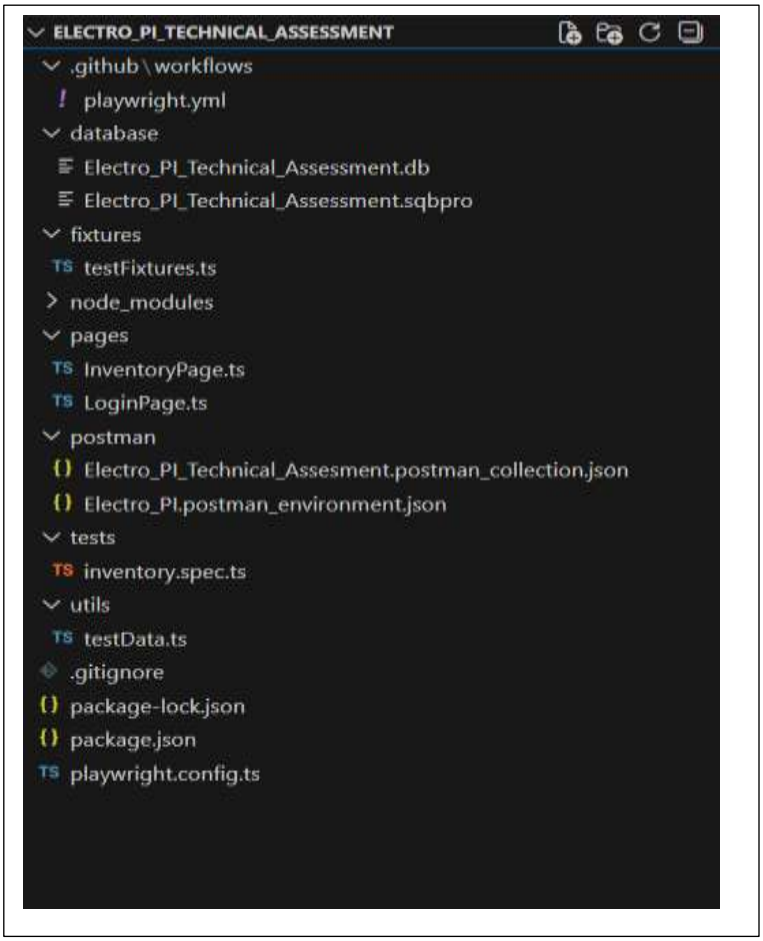
Part 1: UI Test Automation & Architecture

1. I will choose Playwright with TypeScript for this application because Playwright is well suited for web applications that depend on dynamic UI components and asynchronous operations also it provides built in, auto waiting, reliable element locators, browser context isolation, network interception, and support for multiple browsers it also reduces test flakiness by automatically waiting for elements to become actionable instead of relying on hardcoded delays also support parallel execution, tracing, screenshots, videos, and detailed test reports also make debugging failed test easier also TypeScript provides types safety and makes a scalable automation framework easier to maintain.

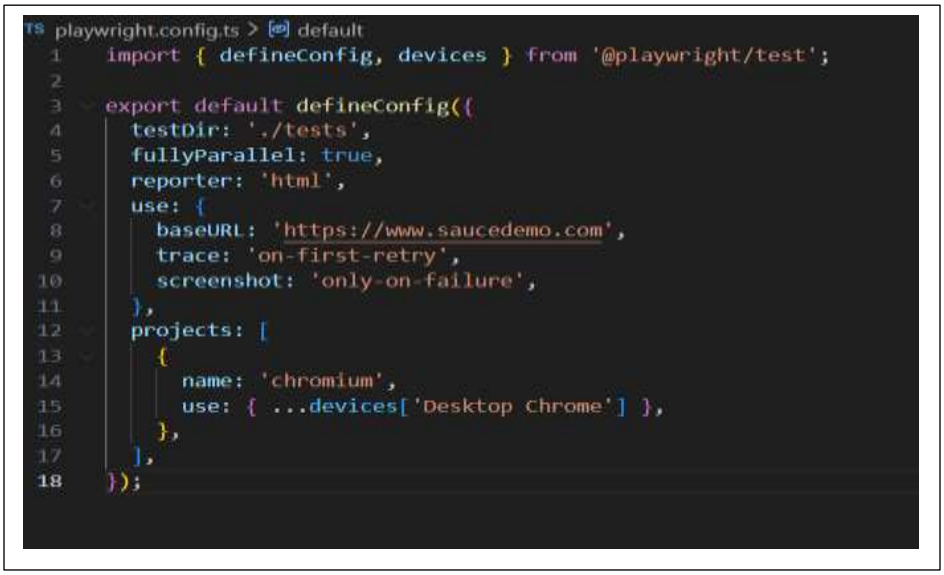
2. I will use the Page Object Model (POM) design pattern combined with a clear separation of test cases, page objects, test data, configuration, and utilities also each page object will contain the locators and actions related to a specific application page, while the test files will contain the actual test scenarios and assertions. This structure will improve:

- Maintainability which means UI changes are isolated within page objects.
- Reusability which means common actions such as login can be reused across multiple tests.
- Scalability which means new modules can be added without affecting existing tests.
- Readability which means tests describe business flows rather than low level UI interactions.
- Debugging which means Playwright traces, screenshots, and structured test reporting can be integrated into the framework.

3. Structure of the Project Implementation will be as follow:



3.1 playwright.config.ts:



3.2 testData.ts:



```ts
export const testData = {
  adminUser: {
    username: 'standard_user',
    password: 'secret_sauce',
  },
  inventoryItem: {
    name: 'Sauce Labs Backpack',
  },
};
```

3.3 LoginPage.ts:



```ts
import { Page, Locator } from '@playwright/test';

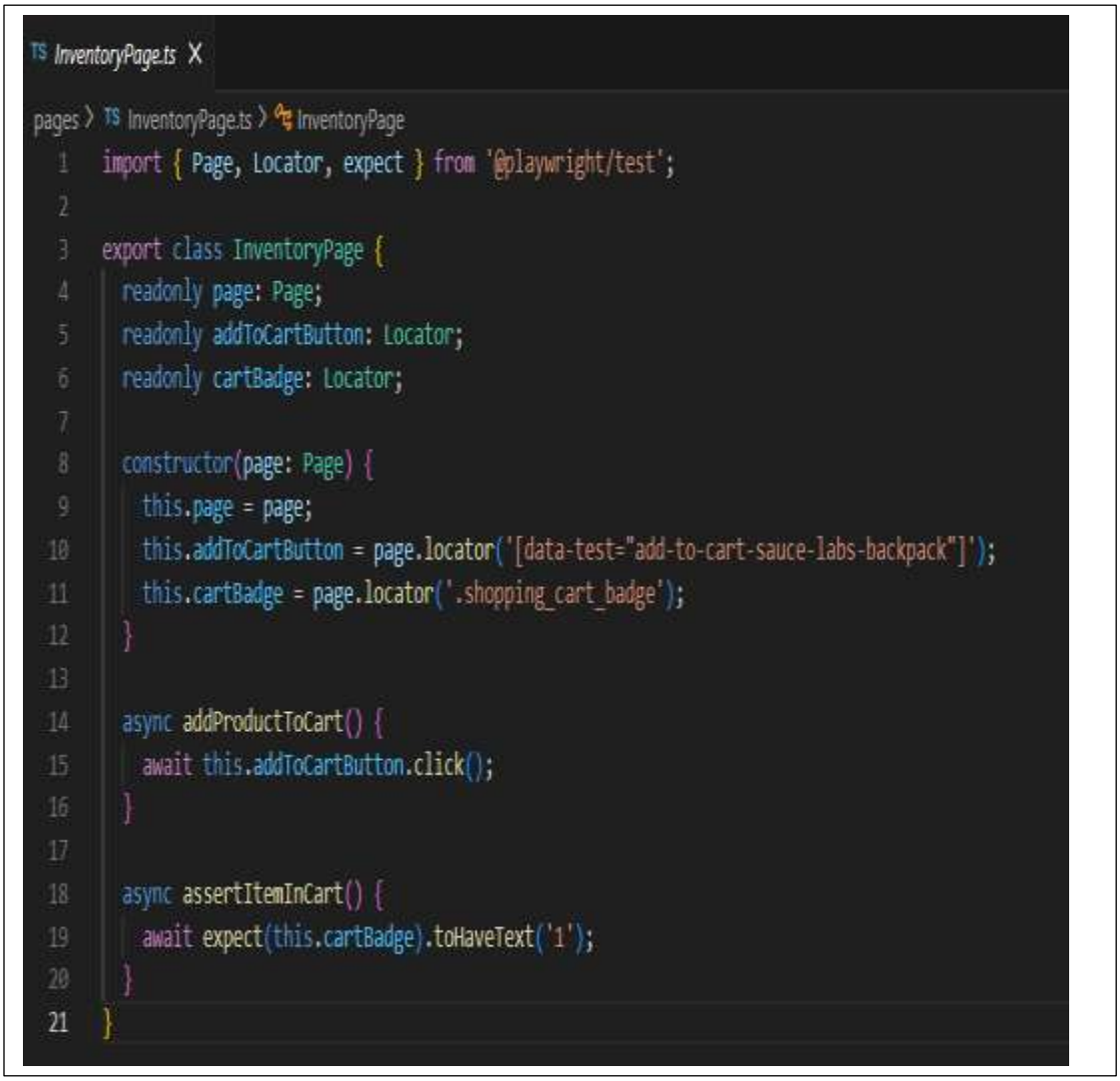
export class LoginPage {
  readonly page: Page;
  readonly usernameInput: Locator;
  readonly passwordInput: Locator;
  readonly loginButton: Locator;

  constructor(page: Page) {
    this.page = page;
    this.usernameInput = page.locator('[data-test="username"]');
    this.passwordInput = page.locator('[data-test="password"]');
    this.loginButton = page.locator('[data-test="login-button"]');
  }

  async navigate() {
    await this.page.goto('/');
  }

  async loginAsAdmin(username: string, password: string) {
    await this.navigate();
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }
}
```

3.4 InventoryPage.ts:



```ts
import { Page, Locator, expect } from '@playwright/test';

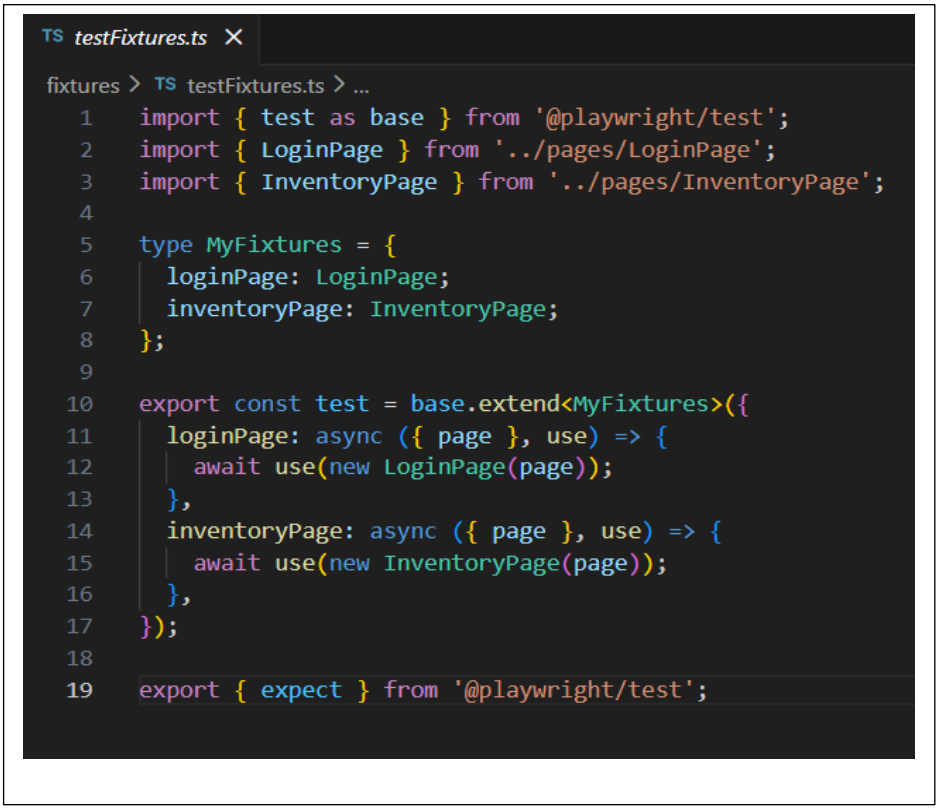
export class InventoryPage {
  readonly page: Page;
  readonly addToCartButton: Locator;
  readonly cartBadge: Locator;

  constructor(page: Page) {
    this.page = page;
    this.addToCartButton = page.locator('[data-test="add-to-cart-sauce-labs-backpack"]');
    this.cartBadge = page.locator('.shopping_cart_badge');
  }

  async addProductToCart() {
    await this.addToCartButton.click();
  }

  async assertItemInCart() {
    await expect(this.cartBadge).toHaveText('1');
  }
}
```

3.5 testFixtures.ts:



```ts
import { test as base } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';
import { InventoryPage } from '../pages/InventoryPage';

type MyFixtures = {
  loginPage: LoginPage;
  inventoryPage: InventoryPage;
};

export const test = base.extend<MyFixtures>({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },
  inventoryPage: async ({ page }, use) => {
    await use(new InventoryPage(page));
  },
});

export { expect } from '@playwright/test';
```

3.6 inventory.spec.ts:



```ts
import { test } from '../fixtures/testFixtures';
import { testData } from '../utils/testData';

test('User can successfully log in and add item to inventory cart', async ({ loginPage, inventoryPage }) => {
  // Uses arguments passed into loginAsAdmin method
  await loginPage.loginAsAdmin(testData.adminUser.username, testData.adminUser.password);

  // Perform inventory action and assertion
  await inventoryPage.addProductToCart();
  await inventoryPage.assertItemInCart();
});
```

4. I will avoid hardcoded sleeps such as **waitForTimeout()**. Instead, synchronization should be based on application state and explicit conditions.

My Steps will go as follow:

- Use Playwright auto waiting and web first assertions.
- Wait for elements to become visible, enabled, or hidden when appropriate.
- Wait for expected navigation or URL changes.
- Wait for relevant network responses when a UI action depends on an API call.
- Use Stable user facing locators such as getByRole and getByLabel instead of fragile CSS/XPath chains.
- Use CI retries only as a safety net retires should not replace proper synchronization.
- Capture traces, screenshots, and reports on failures to simplify debugging.

Part 2: API Testing Strategy

1. Endpoint: POST/api/v1/inventory/items

Authorization: Bearer <token>

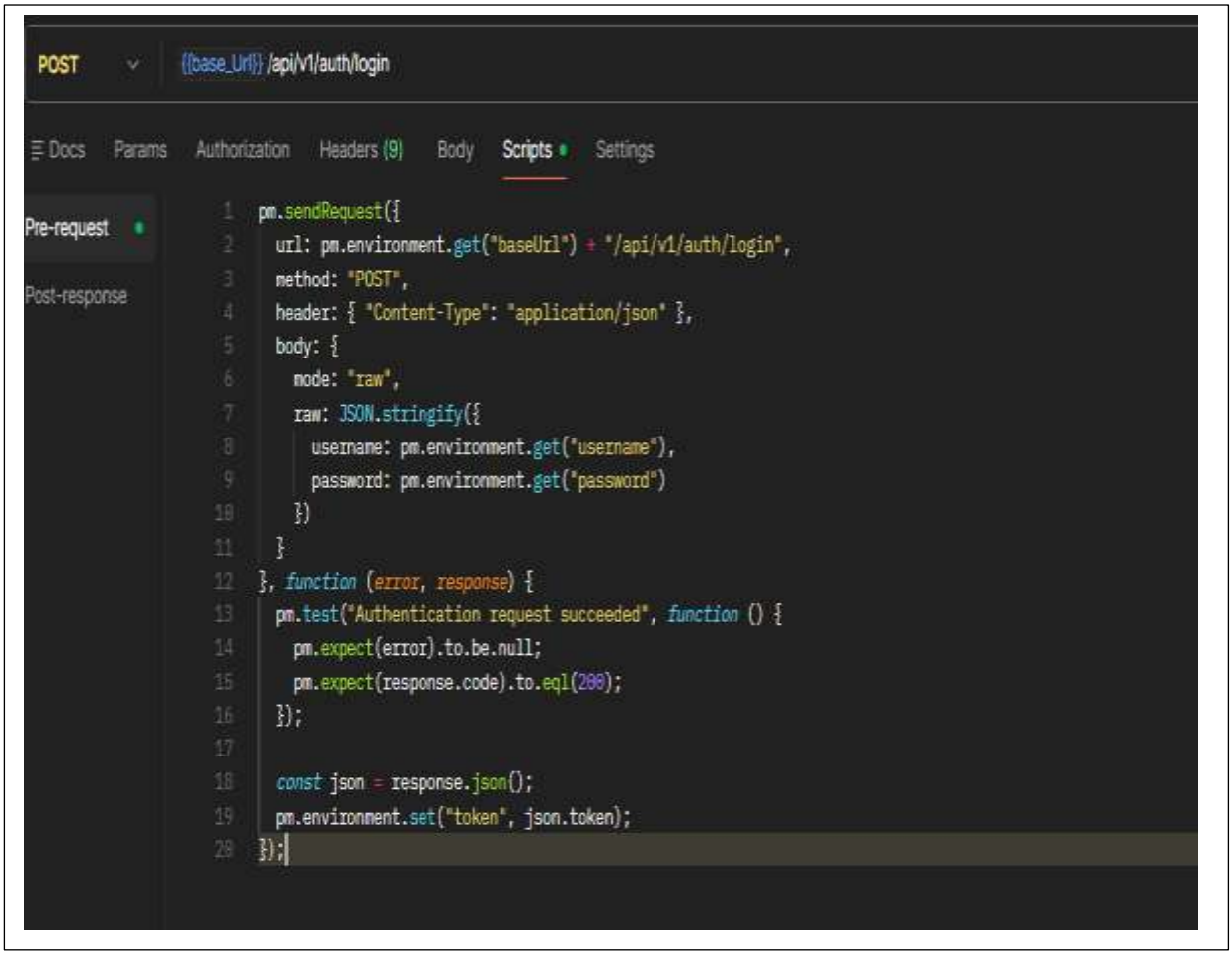
Payload: {“item_name”:“Wireless Mouse”, “sku”:”MS-001”,”quantity”:50,”price”:25.00,”category_id”:3}

2. Test Scenarios:

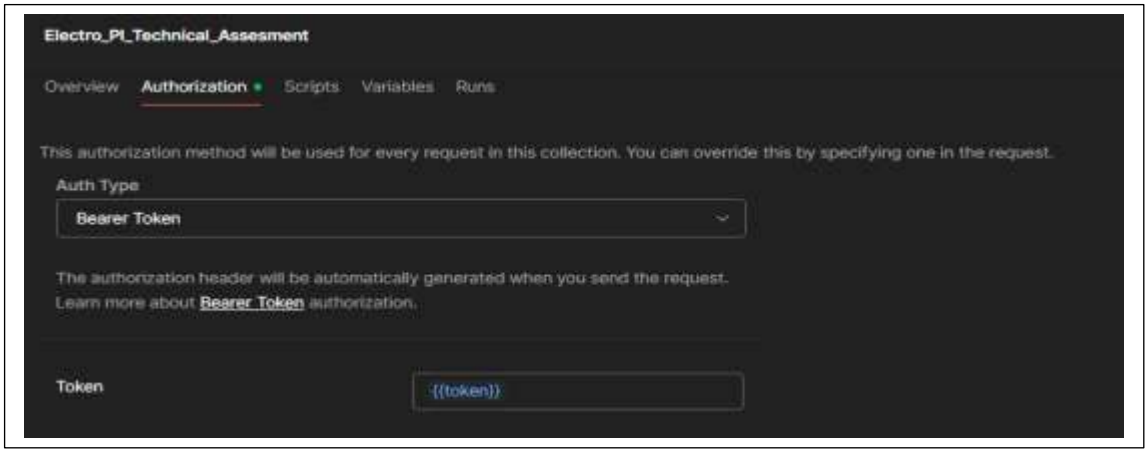
Test Case API Type Method	Scenario	Test Input	Expected Status Code	Expected Result
Positive (POST)	Create valid inventory item	1- Enter valid item_name 2- Enter valid unique sku 3- Enter positive quantity/price 4- Enter valid category_id	201 Created	Item is created and response contains the submitted sku
Positive (POST)	Create another valid item	1- Enter different valid item data 2- Enter valid unique sku	201 Created	Item is created with correct returned data
Negative (POST)	Missing required field	1- Delete sku 2- Delete another required item data field	400 Bad Request	Validation error and item is not created
Negative (POST)	Invalid authentication	1- Enter invalid/expired bearer token	401 Unauthorized	Request is rejected and item is not created
Negative (POST)	Duplicate SKU	1- Enter an sku that already exists	409 Conflict	Request is rejected and items is not created

3. Postman Automation:

A dynamic authentication flow can be implemented in a Postman pre-request script. The Script sends the authentication request, extracts the token from the response, and stores it as an environment variable.



The Inventory request can use:

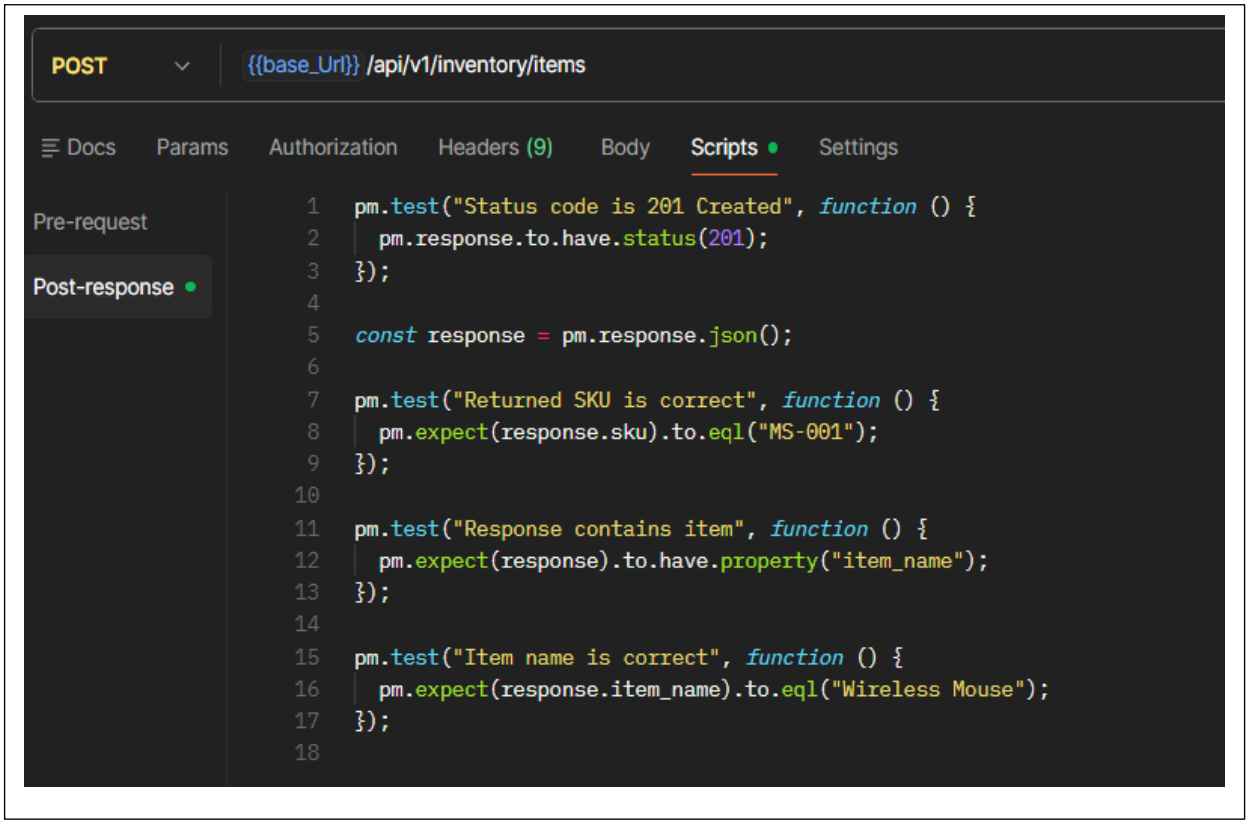


3.1 Postman Assertions:

The Post Method Data (Payload)



The Post Assertions and Test Scripts



Part 3: CI/CD & Database Validation

1. Database Testing:

After the API created the inventory item, the database can be validated by joining the products table with the categories table and filtering by sku and category name.

SQL 1*

```
SELECT p.id, p.item_name, p.sku, p.quantity, p.price, c.name AS category_name FROM products p

INNER JOIN categories c  ON p.category_id = c.id WHERE p.sku = 'MS-001' AND c.name = 'Electronics';
```

	id	item_name	sku	quantity	price	category_name
1	101	Wireless Mouse	MS-001	45	29.99	Electronics

Expected Result showed that the query returns the created items with sku= “MS-001” and category_name= “Electronics” so this validates both record creation and the category relationship.

2. CI/CD Integration:

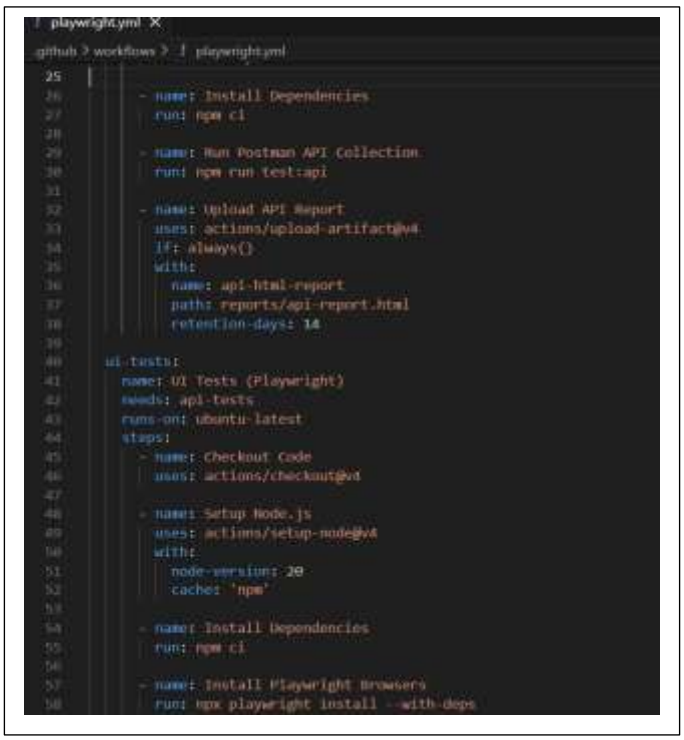
I will integrate the automation suite using GitHub Actions. The pipeline will install dependencies, execute API tests using Newman, executes UI test using Playwright, and publishes test artifacts and reports for both test suites.

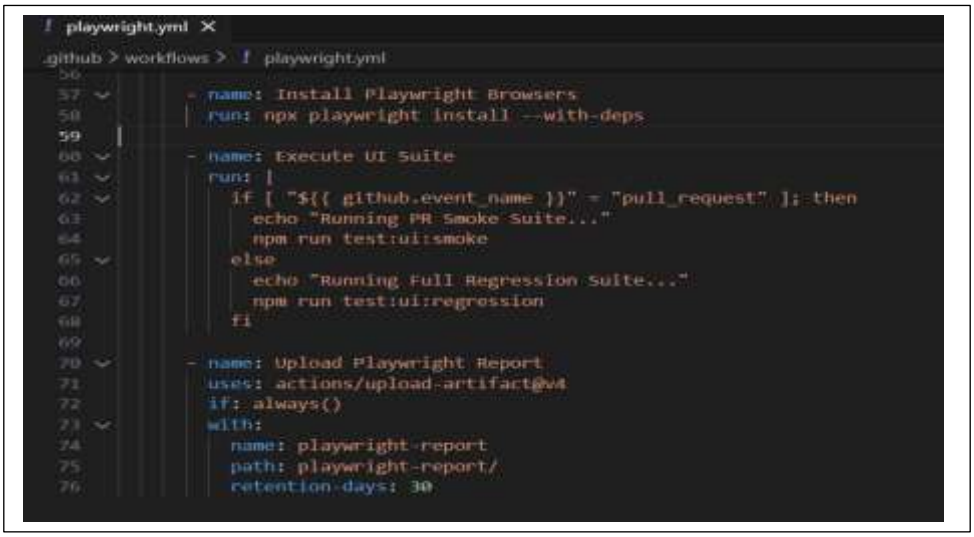
Pipeline work flow steps explain as follow:

Push/ Pull Request / Nightly Schedule -> Install Dependencies -> API Tests -> UI Tests -> Reports / Artifacts -> Pipeline Result.

This Explain how it works in coding (playwright.yml):







3. Execution Strategy:

API Test should run earlier in the pipeline because they are generally faster and provide quick feedback on backend behavior. UI test should run after the relevant API / environment checks and can be expanded into a full regression suite after merge. I will explain how strategy works as follow:

- Pull Request: Run the API suite and focused set of critical UI smoke tests for fast feedback.
- Main branch: Run the full API suite and the broader UI regression suite.
- Scheduled / Nightly execution: Automatically runs the full API and UI regression suite daily at midnight UTC via cron schedule (00***).

cron schedule (00***) means: schedule trigger executes the pipeline automatically every day at midnight (00:00 UTC).

Finally

Click here to get -> GitHub Repository Link

Click here to get -> My GitHub Profile Link